



LeetCode 771: Jewels and Stones

1. Problem Title & Link

Title: LeetCode 771: Jewels and Stones

Link: <https://leetcode.com/problems/jewels-and-stones/>

2. Problem Statement (Short Summary)

You are given:

- `jewels` → string representing types of stones that are jewels.
- `stones` → string representing stones you have.

Return the number of stones that are also jewels.

Each character represents one stone type.

Example

Input:

`jewels = "aA"`

`stones = "aAAbbbb"`

Output:

3

Explanation:

Jewels:

a

A

Stones:

a A A b b b b

Matching jewels:

a

A

A

Count:

3

3. Examples (Input → Output)

Example 1

Input:

`jewels = "aA"`



```
stones = "aAAbbbb"
```

Output:

3

Example 2

Input:

```
jewels = "z"
```

```
stones = "ZZ"
```

Output:

0

Example 3

Input:

```
jewels = "abc"
```

```
stones = "aabbccdd"
```

Output:

6

Example 4

Input:

```
jewels = "A"
```

```
stones = "aAaAa"
```

Output:

2

Because:

'A' != 'a'

Case-sensitive.

4. Constraints

$1 \leq \text{jewels.length} \leq 50$

$1 \leq \text{stones.length} \leq 50$



jewels and stones consist of English letters

All characters in jewels are unique

Important:

Case-sensitive problem.

'a' and 'A' are different.

5. Core Concept (Pattern / Topic)

HashSet

Secondary Topics:

- Membership Lookup
- Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

For every stone:

Check every jewel.

Example:

jewels = "aA"

stones = "aAAbbbb"

For each stone:

a -> compare with a,A

A -> compare with a,A

A -> compare with a,A

...

Works.

Complexity

$O(J \times S)$

where:

J = jewels length



S = stones length

Better Observation

We repeatedly ask:

Is this stone a jewel?

This is a membership lookup problem.

Best Data Structure

Use:

HashSet

because lookup is:

$O(1)$

Example

Jewels:

aA

Store in set:

{'a', 'A'}

Process stones:

a ✓

A ✓

A ✓

b ✗

b ✗

b ✗

b ✗

Count:

3

Key Insight



Convert jewels into a set.

Then every stone can be checked instantly.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Jewels:
aA
Set:
+-----+
| a A |
+-----+

Stones:
a A A b b b b
Check:
a -> Found ✓

A -> Found ✓

A -> Found ✓

b -> Not Found ✗

b -> Not Found ✗
Count:
3
```

8. Pseudocode (Language Independent)

```
create set from jewels

count = 0

for each stone

    if stone in set

        count++

return count
```

9. Code Implementation

Python

```
class Solution:
    def numJewelsInStones(
        self,
        jewels: str,
```



```
stones: str
) -> int:

    jewel_set = set(jewels)

    count = 0

    for stone in stones:

        if stone in jewel_set:
            count += 1

    return count
```

Java

```
import java.util.*;

class Solution {

    public int numJewelsInStones(
        String jewels,
        String stones) {

        HashSet<Character> set =
            new HashSet<>();

        for(char ch :
            jewels.toCharArray()) {

            set.add(ch);
        }

        int count = 0;

        for(char ch :
            stones.toCharArray()) {

            if(set.contains(ch))
                count++;
        }

        return count;
    }
}
```

10. Time & Space Complexity

Time Complexity



$O(J + S)$

Where:

J = jewels length

S = stones length

Space Complexity

$O(J)$

For storing jewels in a set.

11. Common Mistakes / Edge Cases

Mistake 1

Ignoring case sensitivity.

Example:

jewels = "A"

stones = "a"

Answer:

0

not:

1

Mistake 2

Using nested loops.

Works but slower.

Mistake 3

Converting both strings to lowercase.

Changes problem meaning.

Edge Cases

No Jewel Found

Input:

jewels = "z"

stones = "AAAA"

Output:



0

Every Stone Is Jewel

Input:

jewels = "abc"

stones = "abcabc"

Output:

6

Single Character

Input:

jewels = "a"

stones = "a"

Output:

1

12. Detailed Dry Run

Input:

jewels = "aA"

stones = "aAAbbbb"

Set:

{'a', 'A'}

Stone	In Set?	Count
a	Yes	1
A	Yes	2
A	Yes	3
b	No	3
b	No	3
b	No	3
b	No	3

Return:

3



13. Common Use Cases (Real-Life / Interview)

Access Control

Check whether a user belongs to an allowed group.

Filtering

Count elements belonging to a category.

Inventory Systems

Count special items.

Interview Purpose

Tests:

HashSet

Membership Lookup

Counting

14. Common Traps (Important!)

Trap 1

Using List instead of Set.

Lookup becomes:

$O(n)$

instead of:

$O(1)$

Trap 2

Ignoring case sensitivity.

Trap 3

Using nested loops when a set is sufficient.

15. Builds To (Related LeetCode Problems)

LC 771 - Jewels and Stones



LC 217 - Contains Duplicate



LC 349 - Intersection of Two Arrays





LC 202 - Happy Number



LC 128 - Longest Consecutive Sequence

These build HashSet intuition.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Nested Loops	$O(JS)$	$O(1)$	Poor
HashSet	$O(J+S)$	$O(J)$	Best
Frequency Array	$O(J+S)$	$O(52)$	Good

Approach 1: Nested Loops

Compare every stone with every jewel.

Simple but inefficient.

Approach 2: HashSet (Chosen)

Fast lookup.

Most common interview solution.

Approach 3: Frequency Array

Possible because only English letters exist.

Less intuitive than HashSet.

17. Why This Solution Works (Short Intuition)

The problem repeatedly asks:

Is this stone a jewel?

A HashSet answers this question in constant time.

By storing all jewels in a set and checking every stone, we efficiently count all matching stones.

18. Variations / Follow-Up Questions

Follow-Up 1

Return all jewel stones instead of count.

Example:

Output:



[a, A, A]

Follow-Up 2

Count frequency of each jewel.

Example:

a -> 1

A -> 2

Follow-Up 3

Find common characters between two strings.

Related:

LC 349

Intersection of Two Arrays

Follow-Up 4

Support Unicode characters.

HashSet solution still works.

Interview Takeaway

The key pattern is:

HashSet Membership Check

Whenever you hear:

Exists?

Present?

Belongs to a group?

Allowed values?

think:

HashSet

This is one of the most common and powerful interview patterns because it converts repeated searches from:

$O(n)$

to:

$O(1)$

per lookup.